

Department of Electrical and Electronics Engineering

Project Report

Smart Multi-Protection System

Load-wise Fault Isolation

Course no : EEE 4252

Course name : Power System protection lab

Submitted By:

1. Tahomid Uddin (20220105152)
2. Shams Mohammed Suha Chowdhury (20220105172)
3. Farhan Rakeen (20220105173)
4. Ahmed Eyenun Nishat (20220105237)
5. Tausif Naim Jami (20220105245)

Submitted to:

- Mr. Md. Abdullah-Al-Mamun
- Mr. Md.Sadman Amir

Year :4th

Semester :2nd

Section :CD

Objective

The objective of this project is to design and implement a low-cost, Arduino-based smart protection system that provides independent, load-wise fault protection to a multi branch AC distribution setup. The system is intended to:

- Continuously monitor the current drawn by each of four individual load branches using a dedicated current sensor per branch.
- Detect abnormal current conditions in any branch — overcurrent/overload and line-to-ground (short-circuit) faults — through per-branch current sensing.
- Isolate only the faulty branch by tripping its own relay channel, so that the remaining healthy branches continue to operate without interruption (load-wise fault isolation).
- Notify the user remotely of any fault or trip event through an SMS alert sent via a GSM module, and provide a local, on-board LCD read-out of branch status.
- Demonstrate a low-cost, scalable alternative to conventional multi-branch circuit breakers using readily available microcontroller-based components.

Components

The prototype is built around an Arduino UNO, one current sensor and one relay channel per branch (so each branch can be tripped on its own), an LCD for local status display, a GSM module for remote alerts, and two buck converters that supply the 5 V control rail. The complete component list is given below.

No.	Component	Quantity	Role in the System
1	Arduino UNO	1	Central controller — reads all sensor inputs and drives the relay module, LCD and GSM module; powered externally via its DC barrel jack
2	DC Adapter	1	External mains-powered adapter (typically 9–12 V) plugged into the Arduino's barrel jack — the sole power source for the whole board; no battery is used
3	ACS712-5A Current Sensor	4 (1 per branch)	Hall-effect sensor that measures the current drawn by its branch
4	4-Channel Relay Module	1	A single relay board whose four channels are switched independently by separate Arduino pins, giving each branch its own trip
5	GSM Module	1	Sends an SMS alert to the user whenever a branch trips or a fault occurs
6	16×2 LCD Display (I2C interface)	1	Shows live branch status/readings and fault messages directly on the board

No.	Component	Quantity	Role in the System
7	Push Button	1	Manual control input — used to reset a tripped branch once the fault is cleared
8	DC-DC Buck Converter (LM2596-type)	2	Condition power drawn from the Arduino's own supply rail for two downstream loads rather than stepping down a battery: one sits in the GSM module's power path (its onboard electrolytic filter capacitors help absorb the module's transmit-current spikes), and the other supplies the relay module's coil power from the Arduino, regulating it further if required
9	SPST Switch (inline)	8 (2 per branch)	Manually connects Load A (Normal) and Load B (Extra/Overload) in each branch
10	Lamp Holder — 220 V Bulb (Load)	8 (2 per branch)	Fitted with a bulb; represents the Normal load and an additional Overload load in each branch
11	Small breadboard, connecting wires & mounting board	As required	Common ground/power rail distribution and mechanical support; components are wired directly onto a rigid mounting board — no custom PCB is used

Working Principle of the Key Components

- **ACS712-5A Current Sensor:** a Hall-effect current sensor that produces an analog voltage proportional to the current flowing through its branch, allowing the Arduino to compute the actual load current of that branch.
- **4-Channel Relay Module:** a single PCB carrying four electromagnetically operated switches (COM–NO–NC); although the four relays share one board, each is driven by its own Arduino output pin, so each branch is opened/closed completely independently of the others.
- **GSM Module:** a GSM/GPRS modem that communicates with the Arduino over a serial (UART) link using AT commands, and is used here to send an SMS alert whenever a branch trips.
- **16×2 LCD Display:** a two-line character display driven over the I2C bus (via a small backpack adapter), used to show live branch readings and fault/trip messages on the board itself.
- **Power Supply (Buck Converters):** with no battery in the design, the Arduino itself is powered from a separate DC adapter through its barrel jack; the two buck converter modules then condition power drawn from the Arduino's own rail for two downstream loads — one into the GSM module's supply (its onboard electrolytic capacitors helping absorb the current spikes the module draws while transmitting) and one into the relay module's coil supply, regulating it further if required.

Circuit Diagram

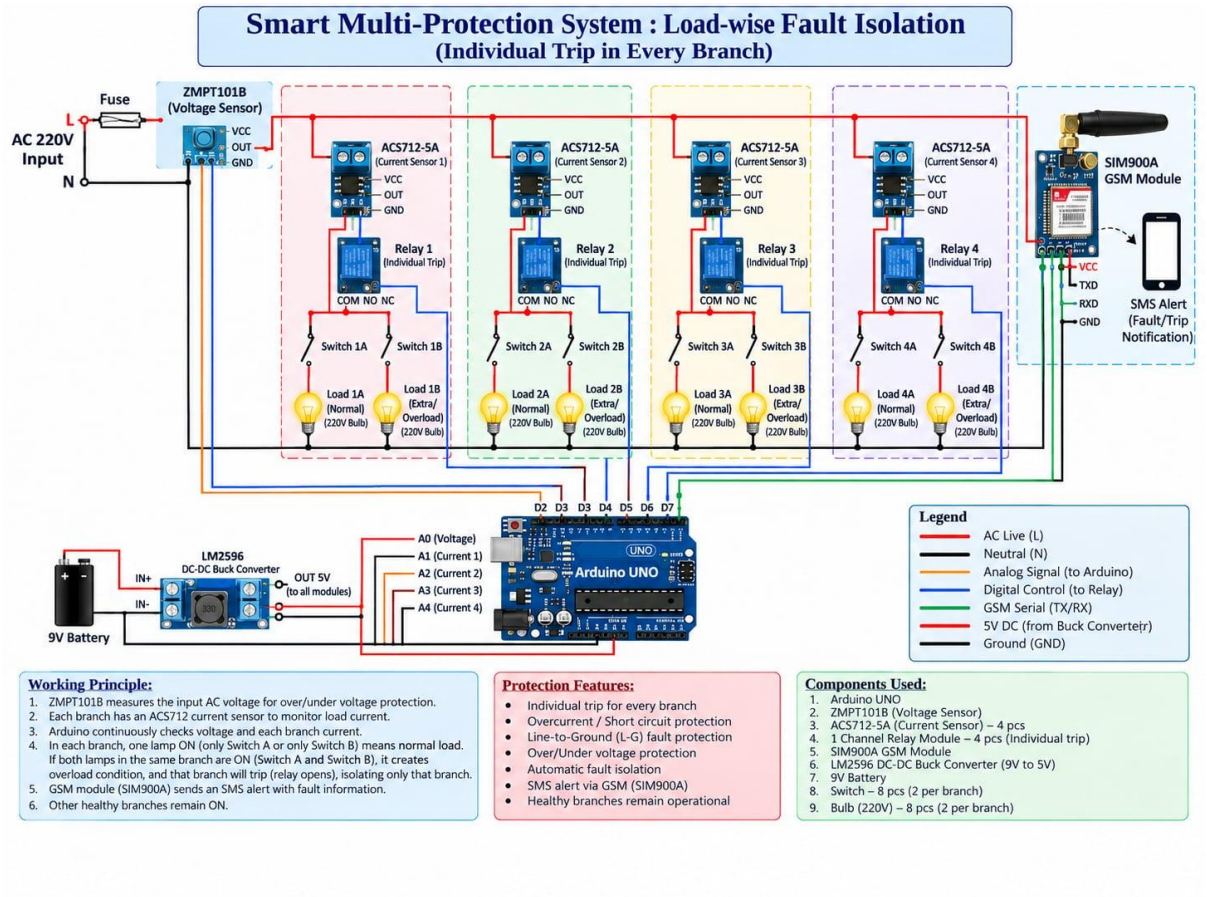


Figure 1: Reference block/circuit diagram of the Smart Multi-Protection System — Load-wise Fault Isolation (original design intent).

Note on the As-Built Prototype

The prototype that was actually assembled and tested (see the photograph in the next section) differs from the reference design of Figure 1 in several respects: (1) the ZMPT101B voltage-sensing stage was omitted, so the system relies entirely on branch-level current sensing rather than also sensing supply voltage; (2) there is no fuse and no battery in the build — the Arduino UNO is instead powered from a separate DC adapter plugged into its barrel jack; (3) two DC-DC buck converter modules condition power drawn from the Arduino's own rail for two downstream loads, rather than a single buck converter stepping down a battery supply — one buck converter sits in the GSM module's power path (its onboard electrolytic capacitors helping absorb the module's transmit-current spikes), and the other supplies the relay module's coil power from the Arduino, regulating it further if required; and (4) the four relay channels are implemented on one 4-channel relay module rather than four separate 1-channel modules. These changes simplify the build while keeping the core load-wise isolation behaviour intact.

Accordingly, in the as-built prototype the AC 220 V supply is distributed directly to four parallel branches, each with its own ACS712-5A current sensor and its own channel on the shared relay module, feeding two switches (e.g., Switch 1A and 1B in Branch 1) that each control a 220 V bulb — one representing the Normal load and one an Extra/Overload load. The four current sensors connect to the Arduino's analog inputs A0–A3, and the four relay channels are driven from four digital output

pins. The GSM module communicates with the Arduino over a serial pair of digital pins, while the 16×2 I2C LCD uses the Arduino's I2C (SDA/SCL) pins and the push-button occupies one further digital input for manual reset. On the power side, the Arduino is powered externally via a DC adapter on its barrel jack; the sensors, LCD and GSM module draw their regulated supply from the Arduino's own rail (with one buck converter conditioning the GSM module's supply), and the relay module's coil supply is likewise drawn from the Arduino through the second buck converter when needed.

Picture of the Project

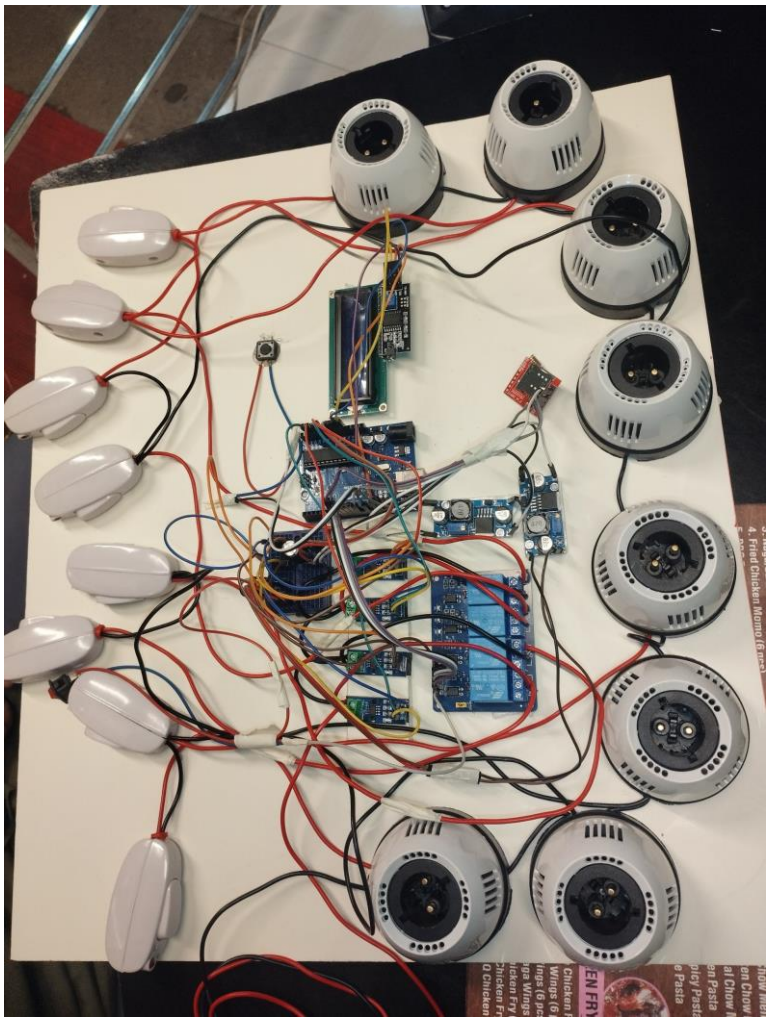


Figure 2: The physically assembled prototype — Arduino UNO with 16×2 I2C LCD and push-button (centre), GSM module and two DC-DC buck converters (right of centre), the four ACS712-5A current sensors and the 4-channel relay module (lower centre), the eight inline SPST switches (left), and the eight branch lamp holders (right/top/bottom).

Working Procedure (Step by Step)

1. Assemble the hardware: wire each of the four branches with its own ACS712-5A current sensor and one channel of the 4-channel relay module (COM–NO–NC), two inline SPST switches, and two 220 V lamp holders fitted with bulbs.
2. Connect the analog outputs of the four ACS712 current sensors to the Arduino UNO's analog inputs A0–A3 (one branch per pin).
3. Connect the four IN pins of the relay module to four Arduino digital output pins; connect the GSM module's TX/RX pins to a pair of digital pins for serial AT-command communication; wire the 16×2 I2C LCD to the Arduino's SDA/SCL pins; and connect the push-button to a spare digital input.
4. Power the Arduino UNO from a separate DC adapter connected to its barrel jack (no battery is used). From the Arduino's own supply rail, route power through the two buck converter modules as needed — one conditioning the supply for the GSM module and one for the relay module's coil supply.
5. Before switching on the AC supply, upload the monitoring-and-protection sketch to the Arduino. The sketch should continuously sample A0–A3, compare each branch's current against a pre-set threshold, and map any threshold-crossing to the corresponding relay channel, an LCD status message, and an SMS command for the GSM module.
6. Power up the control electronics and confirm that the LCD shows a sensible start-up message and that each branch's current reading is visible on the LCD (or the Arduino serial monitor).
7. In each branch, close only Switch A (Normal load) and confirm that the bulb lights and that the reading shown for that branch corresponds to the expected, normal load current.
8. To simulate an overload in a chosen branch, additionally close Switch B (Extra/Overload load) in the same branch so both bulbs are on at once, raising that branch's current beyond its threshold.
9. Verify load-wise fault isolation: only the affected branch's relay channel should open (tripping both of its loads), while the other three branches continue to operate normally and are unaffected.
10. Confirm that, at the moment of the trip, the GSM module sends an SMS alert identifying the tripped branch, and that the LCD shows a corresponding fault message.
11. To test line-to-ground/short-circuit protection, briefly and carefully connect a branch's current-sensor load path to the neutral/ground line; the corresponding relay channel should trip almost instantly due to the high fault current sensed by its ACS712.
12. After clearing a fault, use the push-button to reset the tripped branch and re-close its switches to restore normal operation.

Advantages and Disadvantages

Advantages

- **Independently switched branches:** each of the four relay channels is switched by its own Arduino output pin, so tripping one branch opens only that branch's contacts — the other channels are unaffected even though all four relays sit on one board.
- **Selective tripping:** only the faulty/overloaded branch is disconnected, so healthy branches remain powered and unaffected.
- **Remote awareness:** the GSM module sends an SMS the moment a branch trips, keeping the user informed even when away from the site.
- **Local visibility:** the on-board 16×2 LCD gives an immediate, on-site readout of branch status and fault messages without needing a laptop or serial-monitor connection.
- **Simple manual reset:** a dedicated push-button lets the user reset a tripped branch directly on the board once the fault has been cleared.
- **Low-cost, modular hardware:** built from inexpensive, off-the-shelf Arduino-compatible sensors, a relay module and buck converters rather than costly industrial protection relays.
- **Real-time monitoring:** continuous current sampling across all four branches supports timely detection and logging of abnormal conditions.

Disadvantages

- **Shared relay board:** since all four relays sit on a single PCB, a fault on the shared board (e.g., a damaged trace, connector or power rail) could in principle affect more than one branch, even though each channel is switched independently in software.
- **Limited fault-clearing speed:** electromechanical relays and microcontroller polling are slower than dedicated circuit breakers or solid-state protection devices, particularly for high-speed short-circuit faults.
- **Calibration dependency:** the ACS712 sensors need careful offset/gain calibration for accurate, reliable trip thresholds.
- **No voltage-based protection:** with no voltage sensor fitted, the system cannot detect or protect against over-voltage, under-voltage, or other voltage-only faults on the incoming supply — protection is based purely on branch current.
- **Mechanical relay wear:** electromechanical relays have a finite switching life and can degrade with repeated tripping.
- **GSM dependency:** SMS alerts depend on cellular network coverage and SIM/network availability at the installation site.
- **Safety exposure:** despite the sensing and isolation circuitry, the 220 V AC wiring on the prototype board still demands strict insulation and safety practice.

Limitations

- **No voltage sensing:** the as-built prototype has no voltage sensor, so it cannot detect over-voltage, under-voltage, or voltage-only faults on the incoming line. Suggested fix: reintroduce a voltage-sensing stage (e.g., a ZMPT101B module) feeding a dedicated main relay for whole-board protection, as in the original design intent.
- **Limited fault coverage:** the design is primarily tuned for overload detection; short-circuit clearing is not as fast as a dedicated circuit breaker. Suggested fix: add fast comparator-based short-circuit detection or replace the electromechanical relay module with solid-state relays (SSRs) for near-instant tripping.
- **No backup fuse:** there is no fuse (or other basic overcurrent device) on the incoming AC line, so the current sensors and relay module are the only line of defence — if the electronics fail to respond in time, there is no backup protection. Suggested fix: add an inline fuse or a small MCB ahead of the branches as a basic backup.
- **Sensor drift and noise:** ACS712 readings can drift with temperature and are sensitive to noise on a breadboard prototype. Suggested fix: periodic calibration, software/RC filtering, and shielded sensor wiring.
- **Shared relay board:** since the four relay channels share one PCB, a shared-board fault could affect more than one branch. Suggested fix: move to four separate 1-channel relay modules (as in the original reference design) for full physical isolation, or add per-channel fusing on the shared board.
- **Single point of remote-alert failure:** if the GSM module loses network coverage, no SMS alert is sent even though the protective trip still occurs (the LCD still shows the fault locally). Suggested fix: add a secondary notification path such as Wi-Fi/IoT push notification.
- **Manual reset:** a tripped branch currently requires the on-board push-button to be pressed rather than resetting automatically. Suggested fix: implement an auto-reset-after-fault-clear routine with a retry/lockout counter, keeping the push-button as a manual override.
- **Prototype-scale loads:** the system was tested with resistive bulb loads rather than real appliances with inductive or inrush characteristics, which could otherwise trigger nuisance trips. Suggested fix: add inrush-aware thresholds or a short trip delay/debounce.

Discussion

Testing of the prototype confirmed that the branch current sensors and the shared 4-channel relay module together deliver effective load-wise fault isolation. With only the Normal-load switches closed, all four branches operated simultaneously without interfering with one another, and each branch's live current reading tracked its expected load. When the Extra/Overload switch was additionally closed on a chosen branch, the corresponding ACS712 sensor registered the rise in current and the Arduino opened only that branch's relay channel — the other three channels, driven independently by their own Arduino output pins, continued operating normally. This shows that per-channel current sensing together with independent pin-level relay control is enough to keep branch isolation effectively independent, even with all four relays sitting on one shared PCB rather than on four separate boards.

The GSM alerting and the on-board LCD both performed as intended: each trip produced an SMS identifying the affected branch, while the LCD gave an immediate local read-out of status without requiring a laptop connection. As built, however, the system omits voltage sensing entirely, so over-voltage, under-voltage and other line-voltage faults fall outside its current protection scope — a deliberate simplification of the original design that trades voltage-fault coverage for a smaller, cheaper control stage built around two compact buck converters rather than a dedicated voltage sensor and main relay. Careful calibration of the four ACS712 sensors remained important, since small offset or gain errors directly shift the trip thresholds and can lead to nuisance trips or missed faults.

Overall, the project supports the core idea of low-cost, Arduino-based, load-wise fault isolation with both local (LCD) and remote (SMS) notification as a practical stepping stone toward smarter distribution boards for residential or small commercial use. A more complete version would extend the current build by reintroducing voltage sensing and a dedicated main relay for whole-board protection, adding solid-state switching for faster fault clearing, and automating branch reset — the improvements identified in the Limitations section above.